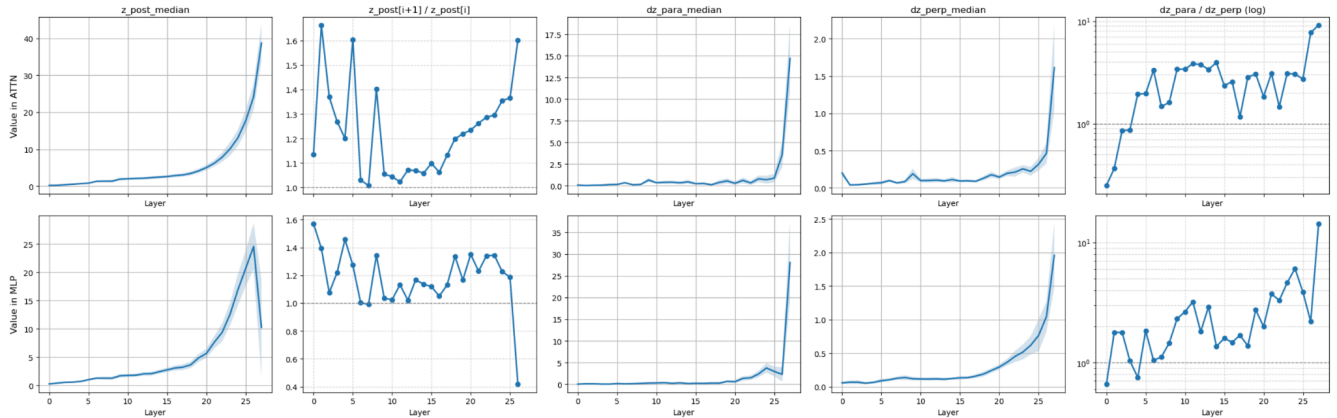


Demystifying Transformers through Representation Geometry: A Unified View of Architectural Foundations and Evolution



Visualization in Logits Space

- ***z_post_median***: Overall representation scale.
- ***z_post[i+1] / z_post[i]***: Layer-wise scale amplification.
- ***dz_para_median***: Parallel updates aligned with the input representation.
- ***dz_perp_median***: Orthogonal updates perpendicular to the input representation.
- ***dz_para / dz_perp (log)***: Dominance of parallel updates over orthogonal change.

Existing Observations or Insights:

1. High latent similarity across layers vs. strong sensitivity of output probabilities to pruning.
2. Progressively increasing activation magnitudes vs. consistently small layer-wise updates.
3. Layer-wise updates predominantly align with the representation direction, whereas updates in orthogonal directions remain limited.
4. Stability (e.g., attention sink, massive activation, normalization) v.s. Transformation (corresponding mitigation)

Why it happens:

Pre-Norm:

$$x_l = x_{l-1} + F(x_{l-1})$$

Understanding the Difficulty of Training Transformers

Additivity of Gaussian distributions: $\text{Var}(x_l + F(x_l)) > \text{Var}(x_l)$

The magnitude of x increases gradually, but that of $F(x_l)$ remains consistent.

How to explain it:

$$z_{l+1} = z_l + \Delta z_{\parallel} + \Delta z_{\perp}$$

$\Delta z_{\parallel}$ acts as a stabilizing component, whereas $\Delta z_{\perp}$ drives representational progress.

A linear combination of one non-zero vector and $(d-1)$ orthogonal vectors can represent a d -dimensional space.

Dominant parallel updates contribute little beyond scaling

$$z_l + \Delta z_{\parallel} + \Delta z_{\perp} = \alpha_l \cdot z_l + \Delta z_{\perp}$$

Exhausting a large volume of computation to $\Delta z_{\parallel}$ is wasting, since it perfectly aligns with the direction of the input. Instead, simply predicting the scale factor is sufficient.

On the other hand, the role of scale factors can be eliminated by the LayerNorm: $\hat{x} = \frac{x - \mu}{\sigma}$. Scaling the input will also scale the mean and variance values, which does not affect output of $\hat{x}$.

Orthogonal updates play essential roles in adjusting the input.

In the embedding or logit space, orthogonal updates cause only minor deviations from the original vectors, suggesting that deeper layers primarily preserve existing representations. Nevertheless, owing to the translation invariance of softmax, absolute shifts in logit magnitude are canceled out, while relative differences introduced by orthogonal components are preserved and subsequently reflected in the output probabilities.

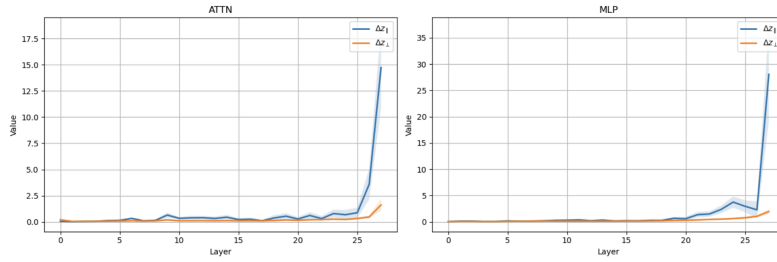
As an illustration, the pre-softmax logits of the top-5 tokens at the 24th layer are shown below:

$x_{in} = [72.76065826416016, 73.730224609375, 73.1319580078125, 72.79354858398438, 71.16820526123047]$

$x_{out} = [74.50714111328125, 74.19922637939453, 73.54593658447266, 72.94505310058594, 71.49601745605469]$

$x_{para} = [0.8439550995826721, 0.8552011847496033, 0.848261833190918, 0.8443366289138794, 0.8254841566085815]$

$x_{\text{perp}} = [0.9025277495384216, -0.386199414730072, -0.4342832565307617, -0.6928321123123169, -0.4976719617843628]$



x_{perp} is more significant in the top tokens, although it is still much lower than the x_{out} and x_{para} , so it can not deviate the vector direction so much. However, such small change can alter the order of top tokens:

This is because softmax has translational invariance, e.g.,
 $\exp(74.50714111328125) / \exp(74.19922637939453) = \exp(0.50714111328125) / \exp(0.19922637939453)$

so in this case, the updates can alter the order of top tokens.

$p_{\text{in_top}}: [0.8176234364509583, 0.1753564178943634, 0.006688379682600498, 0.00033153468393720686, 2.3658544989757502e-07]$

$p_{\text{in_idx}}: [143750, 142690, 141173, 145603, 142585]$

$p_{\text{out_top5}}: [0.9369294047355652, 0.047053012996912, 0.008664444088935852, 0.007350502070039511, 2.5606834697100567e-06]$

$p_{\text{out_idx}}: [142690, 141173, 145603, 143750, 142585]$

Hypothesis:

- Since even small perturbations can substantially alter the probability distribution, the model tends to adopt a conservative strategy when optimizing z_{perp} .

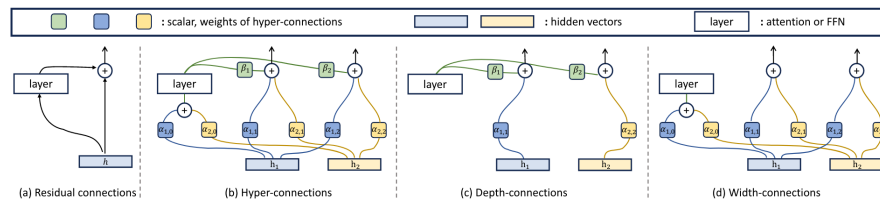
How to utilize / mitigate it (relation to current works):

Focusing expensive computation on meaningful orthogonal updates, while making parallel updates computationally cheaper.

1. Hyper-connection.

mHC: Manifold-Constrained Hyper-Connections

Hyper-Connection



$$z_{l+1} = z_l + (\alpha_l - 1) \cdot z_l + \Delta z_{\text{perp}}$$

when $\alpha_l = 1$, the original residual connection perfectly aligns with the formulation and the residual blocks only have to approach the orthogonal component.

- a. For the current input: scale the parallel components
transfer more parallel updates to the connection branch, and make the residual block more focused on the orthogonal updates.
- b. For cross-layer input: involvement of orthogonal updates
While inputs from different layers may have high similarities, cross-layer input also introduces orthogonal updates.

2. Attention Sink

- a. Attention sinks frequently form around the first or special tokens, drawing attention mass away from semantically meaningful tokens.

Although this may seem suboptimal, it can serve as a stabilizing mechanism: anchoring attention to fixed tokens reduces sensitivity to spurious token interactions. Furthermore, by the deeper layers, token representations are often already well aggregated, rendering further attention mixing redundant and potentially destabilizing.

- Hypothesis:

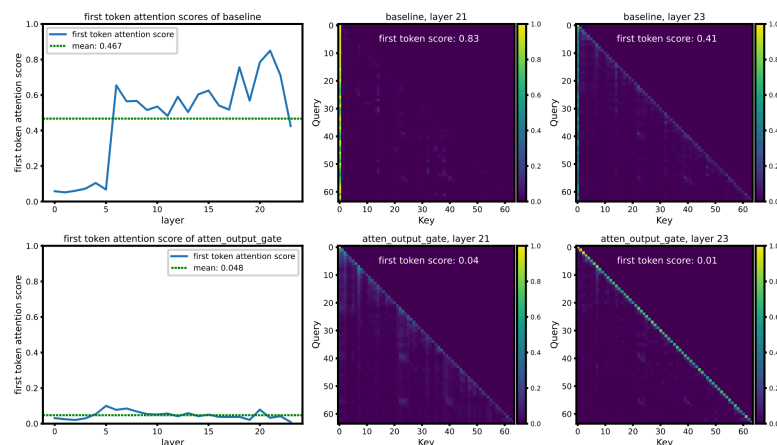
The aggregation from the sunked tokens is more like a bias or parallel updates.

- b. Attention Variant that mitigates attention sink.

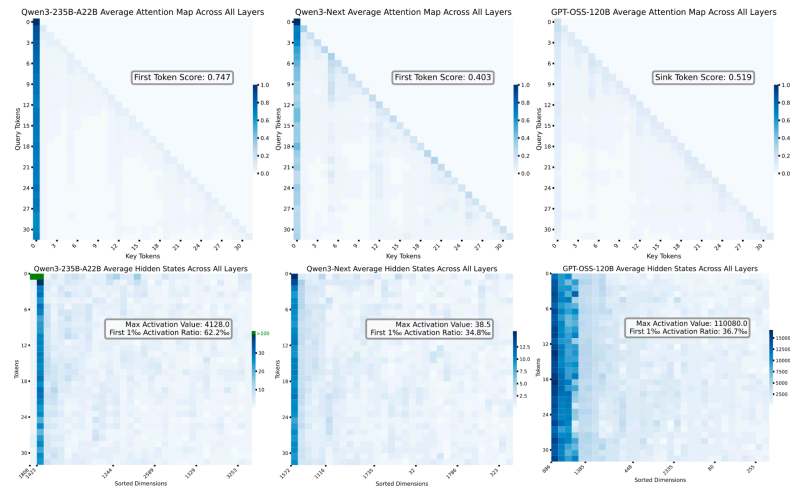
- i. *Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free*

- Hypothesis:

Separate attention to meaningful tokens, which may encourage orthogonal updates.



- ii. *A Unified View of Attention and Residual Sinks: Outlier-Driven Rescaling is Essential for Transformer Training*



Outliers values exist in both Attention and MLP:

Softmax normalization in attention and massive activation in MLP.

Outliers interact with normalization to rescale the non-outlier components after normalization.

Insights and Modifications:

- (1) Directly Clipping or Constraining Outliers Hurts Stability and Performance
- (2) Fusing Outliers into Parameters

$$\text{PreAffineRMSNorm}(\mathbf{x}) = \text{RMSNorm}(\lambda_1 \odot \mathbf{x})$$

- (3) GatedNorm: Explicitly Enabling Rescaling

$$\mathbf{y}_g = \sigma(\mathbf{W}_{\text{up}}(\text{swish}(\mathbf{W}_{\text{down}}(\mathbf{y}))))), \quad \mathbf{y}' = \mathbf{y}_g \odot \mathbf{y}$$

The original modeling entangles multiple functions, combining parallel updates that promote stability with orthogonal updates that refine representations. However, employing complex transformations to achieve parallel updates is inefficient, as similar effects could be realized with simple scaling factors. At the same time, the current formulation lacks an explicit mechanism to prevent this redundancy, motivating the need for more refined modeling strategies.

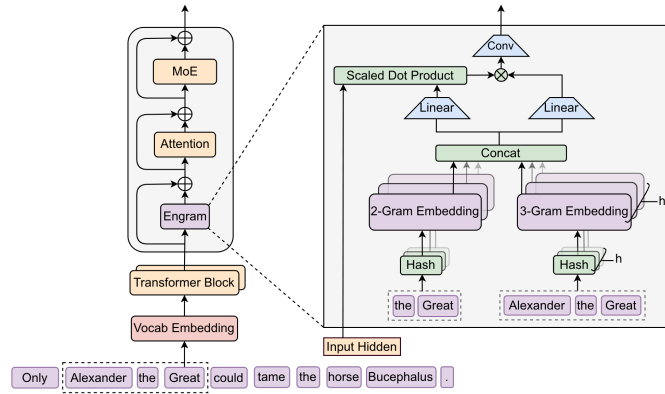
3. Mixture of Experts

Hypothesis:

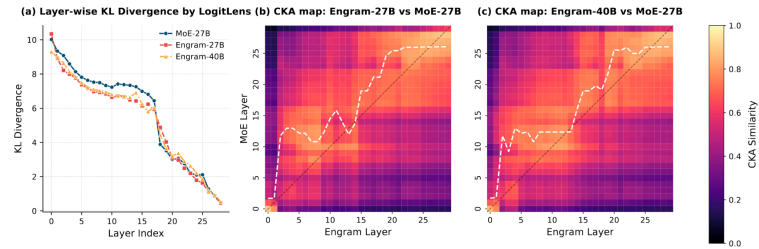
- Each routed expert can provide updates in different orthogonal directions.
- Shared experts perform like a stabilizer for parallel updates.

4. Over-encoding / Discrete Lookup Tables

Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models

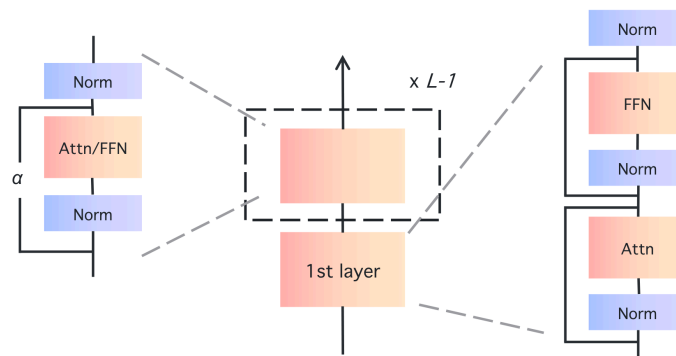


The lookup table also introduces the orthogonal updates, since they are not derived from the input for the current layer. The involvement decreases the layer-wise representation similarities.



5. LayerNorm.

Post-LayerNorm Is Back: Stable, ExpressivE, and Deep



$$\mathbf{x}_{l+1} = \text{LN}(\alpha \mathbf{x}_l + \mathcal{F}_l(\text{LN}(\mathbf{x}_l)))$$

For the backward:

Post-norm encourages diversity but loses the stability of optimization.
The scaling factors enable the decaying gradient for shallow layers.

For the forward:

The scaling factors can adjust the proportion of parallel components.
The additional outer LN introduce more orthogonal updates